

Section 6 — Spring Security Internals & JWT Authentication

The single highest-leverage topic for backend interviews: how Spring Security's filter chain actually works under the hood, how passwords are safely stored, how authentication and authorization are genuinely different, and how to build stateless, production-grade JWT authentication from first principles — token generation, signature validation, and a custom filter wired into the security chain.

Videos 107–129 (23 videos)

~4 hrs runtime

FAANG + Fintech interview focus

Builds on Sections 1–5

01 Hello Spring Security — Your First Handshake with the Framework

Video: Hello Spring Security - Your First Handshake with the Framework

Adding the dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

The moment this single starter is added — with **zero configuration code** — every endpoint in the application becomes protected. This is Spring Boot's auto-configuration philosophy (Section 3) applied to security: `SecurityAutoConfiguration` detects Spring Security on the classpath and activates a fully-functional, secure-by-default setup.

💡 THE PHILOSOPHY: SECURE BY DEFAULT

Spring Security's defaults are deliberately restrictive — every request requires authentication unless you explicitly say otherwise. This "closed by default, opened deliberately" posture is a foundational security design principle (the opposite of "open by

default, restricted as an afterthought," which is how countless real-world vulnerabilities happen when a developer forgets to add a restriction).

02 Zero Config Magic — What Spring Security Gives You by Default

Video: Zero Config Magic - What Spring Security Gives You by Default

Out of the box, with nothing but the starter on the classpath, Spring Boot auto-configures:

- A **generated default password** for a single user named `user`, printed to the console at startup (a fresh UUID every restart).
- Every endpoint requires authentication — any unauthenticated request is redirected to a default **login form** (HTML page) or, for non-browser clients, met with an **HTTP Basic Auth** challenge.
- **CSRF protection** enabled by default for state-changing requests.
- Sensible default security headers (e.g., `X-Content-Type-Options`, `X-Frame-Options`) to guard against common browser-based attacks like clickjacking.

⚠ THE GENERATED PASSWORD IS A DEV-ONLY CONVENIENCE

The console-printed password exists purely so a brand-new Spring Boot + Security project is immediately testable without any setup. It's obviously unsuitable for anything beyond initial exploration — the very next videos (static credentials, then real `UserDetailsService` implementations) exist to replace it.

108, 110 Static User Credentials & Disabling Login Forms / Basic Auth

Videos: Static User Credentials - Locking the Door with Custom Logins · Turning Things Off - Disabling Login Forms & Basic Auth

Static credentials (properties)

Disabling defaults in `SecurityFilterChain`

```
# application.properties – simplest possible custom login,
# still NOT production-appropriate (plaintext, single hardcoded user)
spring.security.user.name=admin
```

```
spring.security.user.password=admin123
spring.security.user.roles=ADMIN
```

! WHY DISABLE THESE AT ALL?

The default HTML login form is meant for traditional server-rendered web apps — irrelevant for a stateless REST API consumed by a separate frontend (or mobile app). HTTP Basic Auth sends credentials in **every single request** (base64-encoded, not encrypted — trivially decodable), which doesn't fit a token-based (JWT) authentication model at all. Disabling both is the standard first step before building a real, stateless, token-based security configuration.

111 Demo of permitAll, denyAll and CSRF Disable

Video: Demo of permitAll, denyAll and CSRF disable

```
authorizeHttpRequests basics
```

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/v1/auth/**").permitAll()    // no authentication requ
        .requestMatchers("/api/v1/admin/**").denyAll()      // blocked for EVERYONE,
        .anyRequest().authenticated()                      // everything else need
    )
    .csrf(csrf -> csrf.disable());    // see explanation below – appropriate for sta
```

WHAT IS CSRF, AND WHY DISABLE IT FOR A REST API?

CSRF (Cross-Site Request Forgery) exploits the browser's automatic inclusion of cookies on every request to a site — a malicious page can trick a logged-in user's browser into submitting a state-changing request (e.g., a hidden auto-submitting form) to your app, and the browser helpfully attaches the victim's session cookie, making it look legitimate. Spring Security's CSRF protection requires a unique, unguessable token on every state-changing request to defeat this.

💡 WHY THIS DOESN'T APPLY TO TOKEN-BASED (JWT) APIS

CSRF is fundamentally a **cookie-based session** vulnerability — it exploits the browser automatically attaching cookies. A stateless API authenticated via a **JWT sent explicitly in the Authorization header** (not a cookie) is not vulnerable to CSRF at all: an attacker's malicious page has no way to make the victim's browser automatically attach that header, since headers (unlike cookies) are never auto-attached by the browser to cross-origin requests. This is precisely why disabling CSRF is the **correct, safe** choice for a stateless

JWT-based REST API — not a security shortcut, but a decision matching the actual threat model.

⚠ NEVER DISABLE CSRF FOR A COOKIE/SESSION-BASED APP

If your app authenticates via a session cookie (the traditional server-rendered or same-origin-cookie-auth model), disabling CSRF genuinely reopens the vulnerability it was designed to close. The safe-to-disable rule applies specifically because JWT-in-header authentication is immune to the attack CSRF protection exists to prevent — not as a blanket rule for all APIs.

Deep Dive — Highest Interview Weight

112- Crafting Your Own Rules — Custom Security 113 Configurations

Videos: Crafting Your Own Rules - Custom Security Configurations Parts 1 & 2

A complete, realistic SecurityConfig

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity // enables @PreAuthorize/@PostAuthorize on service/controller
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless JWT API — safe,
            .cors(Customizer.withDefaults()) // picks up the CorsConfig
            .sessionManagement(sm ->
                sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // no h
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**", "/swagger-ui/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/v1/jobs/**").permitAll() //
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN") // r
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .httpBasic(basic -> basic.disable())
            .formLogin(form -> form.disable());
        return http.build();
    }
}
```

```

    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // covered in depth in the Password E
    }
}

```

METHOD	PURPOSE
<code>hasRole("ADMIN")</code>	Requires the authenticated principal to hold authority <code>ROLE_ADMIN</code> (Spring auto-prefixes <code>ROLE_</code> — don't include it yourself)
<code>hasAuthority("SCOPE_read")</code>	Requires an exact authority string — no auto-prefixing, used for fine-grained permissions/scopes rather than coarse roles
<code>hasAnyRole(...)</code> / <code>hasAnyAuthority(...)</code>	Matches if the principal holds any one of several listed roles/authorities
<code>requestMatchers(HttpMethod.GET, "...")</code>	Scopes a rule to a specific HTTP method, not just a path — common for "public reads, protected writes" patterns

⚠ RULE ORDERING MATTERS — FIRST MATCH WINS

`authorizeHttpRequests` rules are evaluated **in the order declared**, and the **first matching rule applies** — later rules for the same path are never reached. A very common real bug: placing `.anyRequest().authenticated()` *before* a more specific `permitAll()` rule for a sub-path, which makes the specific rule unreachable and unintentionally locks down an endpoint meant to be public. Always order rules from **most specific to least specific**, with `anyRequest()` last.

FAANG

Why `SessionCreationPolicy.STATELESS` specifically for a JWT-based API, and what does it actually change?

+

CORE

What's the difference between `hasRole` and `hasAuthority`, precisely?

+

This revisits Section 4's CORS topic, but configured through **Spring Security's** CORS integration rather than a plain `WebMvcConfigurer` — necessary because once Spring Security is on the classpath, **its filter chain runs before Spring MVC even sees the request**, so a CORS configuration living only in `WebMvcConfigurer` can be overridden or never even reached for security-protected endpoints.

`CorsConfigurationSource` bean

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:4200"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}

// then in the SecurityFilterChain: .cors(Customizer.withDefaults())
// - this tells Spring Security to USE the CorsConfigurationSource bean above
```

⚠ WHY CORS MUST BE CONFIGURED BEFORE SECURITY REJECTS THE REQUEST

Spring Security's CORS integration deliberately runs its own `CorsFilter` **early in the filter chain** — before authentication/authorization checks — specifically so that browser **preflight OPTIONS requests** (which never carry auth credentials, per Section 4's CORS mechanics) aren't rejected with a 401/403 before the browser even gets a chance to see the CORS headers it needs. Without this ordering, every cross-origin request to a secured endpoint would fail at the preflight stage, regardless of how correctly CORS itself was configured.

Deep Dive – Highest Interview Weight

115 What Happens Under the Hood — Spring Security Internals

Video: What Happens Under the Hood - Spring Security Internals

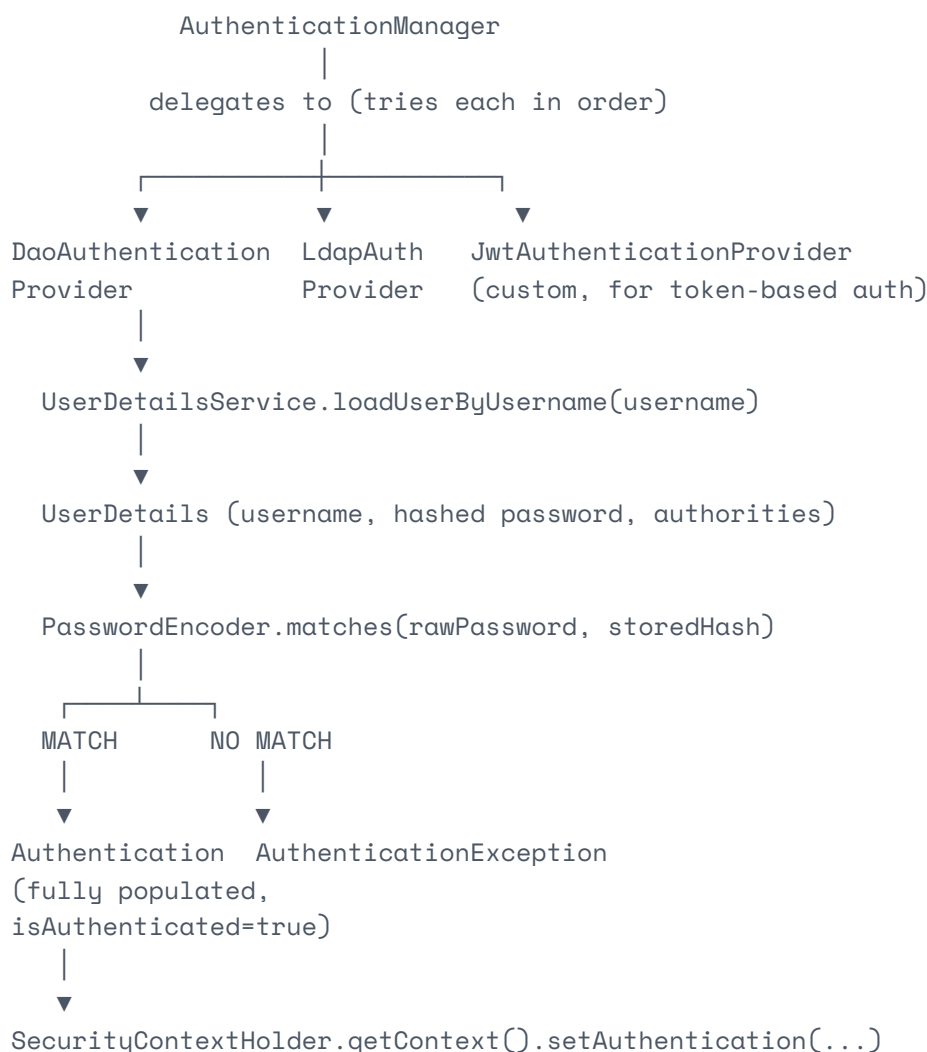
This is the architectural core of the entire section — the mental model every subsequent JWT topic builds directly on top of.



The Core Components & Their Exact Responsibilities

COMPONENT	RESPONSIBILITY
AuthenticationManager	The central orchestrator — receives an unauthenticated <code>Authentication</code> object, delegates to the right <code>AuthenticationProvider</code> , returns a fully-authenticated <code>Authentication</code> object or throws <code>AuthenticationException</code>
AuthenticationProvider	Does the <i>actual</i> authentication logic for one specific mechanism (e.g., <code>DaoAuthenticationProvider</code> checks username/password against a <code>UserDetailsService</code>) — an <code>AuthenticationManager</code> typically delegates across a list of providers, trying each until one succeeds or all fail
UserDetailsService	A single-method interface: <code>loadUserByUsername(String)</code> — your bridge between Spring Security and your actual user data source (database, in-memory, LDAP)

UserDetails	The interface representing an authenticated user's core data — username, password (hashed), granted authorities, and account status flags (enabled, locked, expired)
PasswordEncoder	Verifies a raw submitted password against the stored hash (covered in full detail in the Password Encoders topic)
SecurityContextHolder	A ThreadLocal -based holder for the current request's <code>SecurityContext</code> (which wraps the authenticated <code>Authentication</code> object) — this is how "who is the current user" is accessible anywhere in the call stack without explicitly passing it as a parameter



💡 THE SINGLE MOST IMPORTANT MENTAL MODEL IN THIS ENTIRE SECTION

Spring Security is fundamentally a **chain of Servlet Filters**, not "magic." Every feature — form login, Basic Auth, CSRF, and your own custom JWT validation — is implemented as a filter with a specific, well-defined position in that chain. Once this clicks, the entire

framework stops feeling like a black box: adding JWT support is just "insert one more filter, before the point where Spring would otherwise reject an unauthenticated request."

FAANG

How does SecurityContextHolder make the current user available across an entire request without passing it as a parameter everywhere, and what's the catch with async code?

+

FAANG

What's the difference between authentication and authorization, precisely, and where does each happen in the filter chain?

+

CORE

Why does Spring Security use a Filter chain instead of, say, an interceptor or AOP-based approach like @Transactional does?

+

116-117 Tracing the Security Path — Internal Flow Demo

Videos: [Tracing the Security Path - Internal Flow Demo Parts 1 & 2](#)

A hands-on, debugger-driven walkthrough setting breakpoints inside Spring Security's own source (UsernamePasswordAuthenticationFilter, DaoAuthenticationProvider, ProviderManager) to watch the exact sequence from the diagram above execute in real time against a live login request — turning the architectural diagram into concrete, verified understanding rather than memorized theory.

i WHY THIS HANDS-ON STEP MATTERS FOR INTERVIEWS

Being able to say "I've actually stepped through Spring Security's filter chain in a debugger" is a meaningfully stronger interview signal than reciting the component names from a diagram — it demonstrates the difference between memorized architecture and genuinely internalized understanding, and equips you to debug unfamiliar security configuration issues on the job rather than just describing the theory.

Deep Dive — Highest Interview Weight

118 Encoding vs Encryption vs Hashing

Three fundamentally different operations, frequently confused, and a very common interview question in its own right.

	ENCODING	ENCRYPTION	HASHING
Purpose	Data format transformation for compatibility (e.g., binary → text)	Confidentiality — hide data from unauthorized readers	Integrity/verification — produce a fixed-size fingerprint of data
Reversible?	✅ Yes, trivially — no key needed at all	✅ Yes, but only with the correct key	❌ No — deliberately one-way by design
Needs a key/secret?	❌ No	✅ Yes (symmetric or asymmetric key)	❌ No (though a "salt" is added — not a secret key)
Example	Base64, URL encoding	AES, RSA	BCrypt, SHA-256, Argon2
Used for passwords?	❌ Never — trivially reversible, provides zero security	❌ Rarely for passwords specifically — if the encryption key is ever compromised, every password is instantly recoverable	✅ Yes — the correct and standard approach

⚠️ WHY PASSWORDS ARE HASHED, NEVER ENCRYPTED OR "JUST" ENCODED

Encoding (like Base64) provides **zero security** — it's designed to be reversed by anyone, with no key at all; using it for passwords is equivalent to storing them in plaintext. Encryption is reversible *by design* for legitimate use (decrypting a message meant to be read again later) — but this reversibility is exactly the wrong property for password storage: if a database breach also exposes the encryption key (a very plausible joint compromise), every single user's plaintext password is immediately recoverable. **Hashing is deliberately one-way** — there is no key to steal that would let an attacker recover the original password; the system only ever needs to verify a match (hash the login attempt, compare hashes), never to reconstruct the original.

FAANG

If hashing is one-way, how does a login system verify a password without ever "decrypting" the stored hash?

+

Deep Dive – Highest Interview Weight

119 How Passwords Really Stay Safe — Introduction to Password Encoders

Video: How Passwords Really Stay Safe & introduction to Password Encoders

Why Plain Hashing (Even SHA-256) Isn't Enough

A naive approach — `SHA-256(password)` stored directly — is vulnerable to **rainbow table attacks**: precomputed lookup tables mapping common password hashes back to their plaintext, built once and reused against any leaked database. Fast, general-purpose hash functions like SHA-256 also make **brute-force** attacks computationally cheap for an attacker with GPU/ASIC hardware, since SHA-256 was designed to be *fast* — the opposite of what you want for password hashing specifically.

The Two Properties a Real Password Hashing Algorithm Needs

PROPERTY	WHY IT MATTERS	HOW IT'S ACHIEVED
Salting	Defeats rainbow tables — the same password produces a <i>different</i> hash for every user, since each hash incorporates a unique random value	A random salt is generated per password, and (with BCrypt) stored <i>as part of</i> the resulting hash string itself — no separate salt column needed
Deliberate slowness (work factor)	Makes brute-force/dictionary attacks computationally expensive even with powerful hardware — milliseconds per guess instead of billions of guesses per second	An adjustable "cost factor" / number of iterations, tunable to stay slow even as hardware gets faster over time

BCryptPasswordEncoder

Anatomy of a BCrypt hash

DelegatingPasswordEncoder

@Bean

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder(); // default strength/work factor = 10  
    // higher strength (e.g. 12) = slower hashing = more resistant to brute-force,  
    // but also slower login for legitimate users – a deliberate, tunable trade-off
```

```

}

// registration:
String hashed = passwordEncoder.encode(rawPassword); // a NEW random salt generated
userRepository.save(new User(username, hashed));

// login:
boolean matches = passwordEncoder.matches(rawPasswordAttempt, storedHash);
// matches() extracts the salt FROM storedHash, re-hashes the attempt with
// that same salt, and compares – never needs a separately stored salt column

```

💡 WHY BCrypt (OR ARGON2) OVER GENERIC HASHES — STATED PRECISELY

"BCrypt is a **purpose-built password hashing algorithm** — it automatically handles per-password salting (defeating rainbow tables) and has a tunable, deliberately slow work factor (defeating brute-force at scale), whereas general-purpose hashes like SHA-256/MD5 are designed to be *fast* — the exact opposite property you want for password storage." **Argon2** (winner of the 2015 Password Hashing Competition) is the more modern, even stronger recommendation for new systems, additionally resistant to GPU/ASIC-based cracking via its memory-hardness — worth mentioning as the current best-practice frontier beyond BCrypt.

FAANG

Why does BCrypt's encode() method produce a DIFFERENT output every time for the same input password, and is that a bug?

+

CORE

If BCrypt hashes don't store the salt in a separate column, how does matches() know which salt to use for comparison?

+

FAANG

How would you safely migrate an existing user base from an old, weak hashing scheme (e.g., unsalted MD5) to BCrypt without forcing every user to reset their password?

+

120–121 Creating Users with InMemoryUserDetailsManager

Videos: Creating users using InMemoryUserDetailsManager Parts 1 & 2

InMemoryUserDetailsManager bean

```

@Bean
public UserDetailsService userDetailsService(PasswordEncoder encoder) {
    UserDetails admin = User.withUsername("admin")
        .password(encoder.encode("admin123"))
        .roles("ADMIN")           // stored internally as authority "ROLE_ADMIN"
        .build();

    UserDetails user = User.withUsername("nikhil")
        .password(encoder.encode("pass123"))
        .roles("USER")
        .build();

    return new InMemoryUserDetailsManager(admin, user);
}

```

InMemoryUserDetailsManager is a full UserDetailsService implementation backed purely by an in-memory collection — no database at all. It's an excellent tool for **demos, prototyping, and understanding the UserDetailsService contract in isolation**, but obviously unsuitable for production (users vanish on every restart, no persistence, no scaling across multiple instances).

⚠ THE PRODUCTION PATH: A CUSTOM USERDETAILSSERVICE

Real applications implement UserDetailsService themselves, backed by a JPA repository (from Section 4) — `loadUserByUsername()` queries the database for the user, maps the result to a UserDetails implementation (often by having your own User entity implement UserDetails directly, or via a dedicated adapter class), and returns it. Everything else in the authentication flow (password matching, authority checking) works identically to the in-memory version — InMemoryUserDetailsManager exists purely to teach the contract without the added complexity of a real data layer.

122–123 Invoking the Spring Security Authentication Flow Manually

Videos: Invoking the Spring Security authentication flow manually Parts 1 & 2

Manual login endpoint

```

@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor

```

```

public class AuthController {

    private final AuthenticationManager authenticationManager;

    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody LoginRequest request) {

        // step 1: build an UNAUTHENTICATED token from raw credentials
        Authentication authRequest =
            new UsernamePasswordAuthenticationToken(request.getUsername(), request.

        // step 2: hand it to the AuthenticationManager – this is where ALL the
        // internal machinery from the "Internals" topic actually executes:
        // UserDetailsService lookup + PasswordEncoder.matches(), end to end
        Authentication authResult = authenticationManager.authenticate(authRequest)
        // throws BadCredentialsException / DisabledException / etc. on failure –
        // caught centrally by @RestControllerAdvice (Section 5)

        // step 3: authentication succeeded – generate and return a JWT
        // (the exact next topic in this section)
        String token = jwtService.generateToken(authResult);
        return ResponseEntity.ok(token);
    }
}

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration conf
    return config.getAuthenticationManager(); // exposes Spring's auto-configured
}

```

💡 WHY THIS MANUAL INVOCATION MATTERS — THE BRIDGE TO JWT

This is the precise architectural seam where a traditional, form-login-based Spring Security app becomes a modern, **token-issuing** API. Rather than letting Spring Security's own UsernamePasswordAuthenticationFilter handle a login form submission and establish a session, you build a **plain @RestController login endpoint** that manually drives the exact same underlying

AuthenticationManager/UserDetailsService/PasswordEncoder machinery — but on success, instead of creating a session, you issue a stateless JWT that the client will present on every subsequent request. Everything from the internals topic still applies; only the final step (session vs token) changes.

What's the exact difference between the

FAANG UsernamePasswordAuthenticationToken passed INTO authenticate() versus the

Deep Dive – Highest Interview Weight

124 **JWT Tokens Explained — Modern Token-Based Authentication**

Video: JWT Tokens Explained - Modern Token-Based Authentication

Session-Based vs Token-Based Authentication

	SESSION-BASED (TRADITIONAL)	TOKEN-BASED (JWT)
Server state	Server stores session data (in memory or a shared store) keyed by a session ID	Stateless — server stores nothing; the token itself carries all necessary claims
Client stores	A session ID, typically via cookie	The full JWT, typically sent in the Authorization: Bearer <token> header
Scaling	Requires sticky sessions or a shared session store (Redis) across server instances	Any server instance can validate any token independently — trivially horizontally scalable
Revocation	Trivial — just delete the server-side session	Hard — a token is valid until it expires; genuine revocation needs an extra mechanism (blocklist/short expiry)
CSRF exposure	Vulnerable (cookie-based) — needs CSRF tokens	Not vulnerable, if kept out of cookies (Section 6's CSRF discussion)

Anatomy of a JWT — Three Base64URL-Encoded Segments

eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJuaWtoaWwiLCJyb2xlIjoiQURNSU4ifQ . 5mZ8...signature..

HEADER

algorithm + token type

.

PAYLOAD

claims: user data (sub, role, exp, iat...)

.

SIGNATURE

cryptographic proof the header+payload weren't tampered with

DECODED HEADER:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

DECODED PAYLOAD:

```
{
  "sub": "nikhil",
  "role": "ADMIN",
  "iat": 1710000000,
  "exp": 1710003600
}
```

SEGMENT	CONTAINS	ENCRYPTED?
Header	Signing algorithm (alg) and token type (typ)	❌ No — just Base64URL <i>encoded</i> , not encrypted
Payload (Claims)	Arbitrary data about the subject — standard claims (sub, iat, exp) plus custom claims (roles, permissions)	❌ No — anyone can decode and read it; never put secrets here
Signature	A cryptographic signature over the header+payload, using a secret key (HMAC) or private key (RSA)	N/A — this is what provides integrity , not confidentiality

⚠️ THE SINGLE MOST COMMON JWT MISCONCEPTION

A JWT is signed, not encrypted. Anyone who intercepts a JWT — a network sniffer, a malicious browser extension, or simply pasting it into jwt.io — can trivially **read** the header and payload; Base64URL encoding is reversible by design, exactly like plain Base64 encoding discussed in the Encoding-vs-Encryption topic. The signature only guarantees the token **hasn't been tampered with** since it was issued — it does nothing to hide the payload's contents. Never place sensitive data (passwords, SSNs, secret internal flags) in a JWT payload.

FAANG

Why is JWT revocation fundamentally harder than session invalidation, and how would you handle "logout" or a compromised token in a real system?

+

CORE

What are the standard registered claims (iat, exp, sub, etc.) and why use them instead of just custom claim names?

+

Deep Dive — Highest Interview Weight

125 How JWT Tokens Are Validated Using the Signature

TOKEN GENERATION (server, at login):

`base64url(header) + "." + base64url(payload)`



`signature = HMAC-SHA256( header.payload , SECRET_KEY )`



`final token = header.payload.signature → sent to client`

TOKEN VALIDATION (server, on every subsequent request):

`incoming token split into header.payload.signature`



`recompute: HMAC-SHA256( header.payload , SAME SECRET_KEY )`



`compare recomputed signature to the signature IN the token`



`MATCH` `NO MATCH`



Token is authentic –
also check
exp/iat claims



Token was tampered with (or forged, or wrong key) –
REJECT immediately, throw `SignatureException`

HMAC (Symmetric) vs RSA (Asymmetric) Signing

	HMAC (E.G. HS256)	RSA / ECDSA (E.G. RS256)
Key model	One shared secret key — same key both signs and verifies	Key pair — private key signs, public key verifies
Who can verify?	Only whoever holds the shared secret	Anyone with the public key — including entirely separate services
Who can issue new tokens?	Anyone with the shared secret (same party that can verify)	Only the private-key holder — verifiers cannot forge new valid tokens
Best fit	A single monolithic app/service issuing and validating its own tokens	Microservices — an auth service signs with its private key; every other downstream service verifies using only the public key, without ever needing the ability to forge tokens themselves

💡 WHY ASYMMETRIC SIGNING MATTERS FOR MICROSERVICES — A COMMON SYSTEM DESIGN FOLLOW-UP

In a microservices architecture, distributing a **shared HMAC secret** to every service that needs to validate tokens means every one of those services could also *forge* valid tokens — a significant blast radius if any single service is compromised. With **RSA/ECDSA**, only the central Auth service holds the private signing key; every downstream microservice only needs the (non-secret) public key to verify signatures, and a compromised downstream service gains no ability to mint its own valid tokens. This asymmetry is precisely why RS256 is the standard choice in real-world microservices and OAuth2/OIDC systems (a natural bridge to discuss if asked about scaling this pattern).

FAANG

An attacker intercepts a valid JWT and tries to modify the payload's "role" claim from USER to ADMIN, then re-sends it. Walk through exactly why this fails.

+

FAANG

What is the "alg: none" JWT vulnerability, and how does a well-implemented validation library defend against it?

+

126–127 Generating JWT Tokens in the Backend App

Videos: Generating JWT token in backend app Parts 1 & 2

JwtService — generation application.properties — secret & expiry

```
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;    // externalized — NEVER hardcoded in source

    @Value("${jwt.expiration-ms}")
    private long expirationMs;

    public String generateToken(Authentication authentication) {
        String username = authentication.getName();
        String roles = authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.joining(","));

        Date now = new Date();
        Date expiry = new Date(now.getTime() + expirationMs);
```

```

        return Jwts.builder()
            .subject(username)
            .claim("roles", roles)           // custom claim
            .issuedAt(now)
            .expiration(expiry)
            .signWith(getSigningKey())      // HMAC key derived from the secret
            .compact();
    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}

```

⚠ SIGNING KEY REQUIREMENTS FOR HS256

The JWT spec's HMAC-SHA256 requires a signing key of **at least 256 bits (32 bytes)** — a short, weak, or guessable secret string dramatically weakens the entire signature guarantee, since a short key is brute-forceable. Always generate a cryptographically random secret of sufficient length, store it in a proper secrets manager (AWS Secrets Manager, HashiCorp Vault, or at minimum environment variables — never hardcoded or committed to git), and rotate it periodically as a security best practice.

FAANG

Why put roles/authorities directly in the JWT payload instead of looking them up from the database on every request?

+

Deep Dive – Highest Interview Weight

128–129 Building a Filter to Validate the JWT Token

Videos: Building a filter on the backend to validate the JWT token Parts 1 & 2

This is where the Internals topic's filter-chain diagram becomes real, working code — a custom filter, inserted at exactly the point discussed earlier (`.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)`), that runs on **every single incoming request** to validate any presented JWT and populate `SecurityContextHolder` accordingly.

JwtAuthFilter.java – the complete filter

```

@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter { // guarantees exactly ON

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException

        String authHeader = request.getHeader("Authorization");

        // no token, or wrong format – just pass through; let downstream
        // authorization rules (authorizeHttpRequests) decide if that's OK
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7); // strip "Bearer " prefix

        try {
            String username = jwtService.extractUsername(token); // parses + validat

            // only proceed if we have a username AND nothing is already authenticated
            // (avoids redundant work if some earlier mechanism already set it)
            if (username != null && SecurityContextHolder.getContext().getAuthentic

                UserDetails userDetails = userDetailsService.loadUserByUsername(use

                if (jwtService.isTokenValid(token, userDetails)) { // checks signa

                    UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                            userDetails, null, userDetails.getAuthorities());
                    authToken.setDetails(
                        new WebAuthenticationDetailsSource().buildDetails(request))

                    // THIS is the line that makes the request "authenticated" from
                    // here on – everything downstream (controllers, @PreAuthorize)
                    // this user as the current principal
                    SecurityContextHolder.getContext().setAuthentication(authToken)
                }
            }
        } catch (JwtException ex) {
            // invalid/expired/tampered token – do NOT throw; simply don't authenti

```

```

        // downstream authorizeHttpRequests rules will correctly reject as unau
    }

    filterChain.doFilter(request, response); // ALWAYS continue the chain – th
}
}

```



💡 WHY ONCEPERREQUESTFILTER, SPECIFICALLY

`OncePerRequestFilter` is a Spring-provided base class guaranteeing the filter's logic executes **exactly once per request**, even in scenarios involving internal request forwarding/dispatching within the servlet container where a naive raw `Filter` might otherwise run multiple times for the same logical request. Since JWT validation involves real computational work (signature verification, a potential database lookup via `UserDetailsService`), running it more than once per request would be both wasteful and a subtle source of bugs — this is why it's the standard, idiomatic base class for custom Spring Security filters.

⚠ WHY THE FILTER NEVER THROWS ON AN INVALID TOKEN

Notice the filter **catches `JwtException` and simply continues the chain** rather than immediately rejecting the request itself. This is deliberate separation of concerns: this filter's *only* job is authentication (populating who the user is, if determinable) — **authorization** (deciding whether the request is allowed to proceed) is left entirely to the later `AuthorizationFilter`, which evaluates the `authorizeHttpRequests` rules. This keeps the two concerns cleanly separated exactly as established in the Internals topic, and correctly allows requests to endpoints marked `permitAll()` to succeed even with no token, an expired token, or a malformed token present.

FAANG

Why does the filter check "if `getAuthentication() == null`" before setting a new one — what scenario is this actually guarding against?

+

FAANG

Trace the complete end-to-end flow: a client logs in, receives a JWT, then calls a protected endpoint. Describe every step across both requests.

+

🔥 Rapid-Fire Round

Master Interview Q&A Bank — Section 6

Spring Security and JWT questions are asked in nearly every mid-to-senior Java backend interview. These synthesize concepts across the whole section.

FAANG

Design a complete, production-appropriate authentication and authorization architecture for a new Spring Boot microservices system from scratch.

+

FAANG

Where should a JWT be stored on the client — `localStorage`, `sessionStorage`, or a cookie — and what are the security trade-offs of each?

+

CORE

What is `@PreAuthorize`, and how does it relate to the `authorizeHttpRequests` rules configured in `SecurityFilterChain`?

+

FAANG

How would you implement a refresh token flow alongside short-lived access tokens?

+

CORE

What HTTP status code should an expired/invalid JWT produce, and how does that get wired up given the JwtAuthFilter doesn't throw exceptions itself?

+

FAANG

Explain the complete difference between BCrypt password hashing and JWT signature verification — both use cryptographic algorithms, but for fundamentally different purposes. Don't confuse them.

+

i NEXT SECTION

Paste the next section's video list whenever you're ready, and I'll build the next HTML notes doc in this same design system.